



DAY 16: FUNCTION OVERLOADING & FUNCTION OVERRIDING IN JAVASCRIPT



Lakshmiprasanna Vakada

Function Overloading in JavaScript

Function Overloading is a feature in many languages where multiple functions can have the same name but different parameter lists.

However, JavaScript does NOT support function overloading in the traditional sense. Instead, we handle it using optional parameters, the arguments object, or rest parameters.

✓ Example 1: Function Overloading (Not Supported in JS)

```
function greet() {  
    console.log("Hello!");  
}
```

```
function greet(name) {  
    console.log("Hello, " + name + "!");  
}
```

```
greet(); // Output: Hello, undefined!  
greet("Alice"); // Output: Hello, Alice!
```

- ◆ In JavaScript, the last defined function will overwrite the previous ones.

✓ Example 2: Function Overloading Using arguments

We can simulate function overloading using the arguments object.

```
function greet() {  
  if (arguments.length === 0) {  
    console.log("Hello!");  
  } else if (arguments.length === 1) {  
    console.log("Hello, " + arguments[0] + "!");  
  } else {  
    console.log("Hello, " + arguments[0] + "! You  
are " + arguments[1] + " years old.");  
  }  
}
```

```
greet(); // Output: Hello!  
greet("Alice"); // Output: Hello, Alice!  
greet("Alice", 25); // Output: Hello, Alice! You  
are 25 years old.
```

- ◆ The arguments object allows us to check how many parameters were passed and handle them dynamically.

✓ Example 3: Function Overloading Using Rest Parameters

The modern ES6 way to handle function overloading is using rest parameters (...args).

```
function sum(...numbers) {  
    return numbers.reduce((acc, num) =>  
acc + num, 0);  
}
```

```
console.log(sum(2, 3)); // Output: 5  
console.log(sum(1, 2, 3, 4, 5));  
// Output: 15
```

- ◆ Using ...args, we can handle a variable number of arguments efficiently.

Function Overriding in JavaScript

Function Overriding occurs when a child class provides a specific implementation of a method that is already defined in the parent class.

✓ Example: Function Overriding in JavaScript (with Classes)

```
class Parent {  
  greet() {  
    console.log("Hello from Parent!");  
  }  
}  
class Child extends Parent {  
  greet() {  
    console.log("Hello from Child!");  
  }  
}  
const obj = new Child();  
obj.greet(); // Output: Hello from Child!
```

The greet() method in the Child class overrides the one in the Parent class.

✓ Example: Function Overriding in JavaScript (with Prototypes)

```
function Person() {}
```

```
Person.prototype.sayHello = function () {  
  console.log("Hello from Person!");  
};
```

```
function Student() {}
```

```
Student.prototype = Object.create(Person.prototype);  
Student.prototype.sayHello = function () {  
  console.log("Hello from Student!");  
};
```

```
const student = new Student();  
student.sayHello(); // Output: Hello from Student!
```


We override the sayHello() method using JavaScript's prototype system.

◆ Key Differences: Function Overloading vs. Function Overriding

Feature	Function Overloading	Function Overriding
Definition	Multiple functions with the same name but different parameters	A child class provides a new implementation of a parent method
Support in JS	Not directly supported	Fully supported
Usage	Simulated using arguments or ...rest	Uses extends in classes or prototype-based inheritance
Scope	Happens within the same function scope	Happens in a class hierarchy

◆ Common Interview Questions on Overloading & Overriding

✓ Question 1: What will be the output?

```
class A {  
    show() {  
        console.log("Class A");  
    }  
}  
class B extends A {  
    show() {  
        console.log("Class B");  
    }  
}  
const obj = new B();  
obj.show();
```

✓ Answer:

Class B

- ◆ The show() method in class B overrides the one in class A.

✓ Question 2: How can you implement function overloading in JavaScript?

✓ Answer:

- 1 Using the arguments object
- 2 Using rest parameters (...args)
- 3 Using default parameters